

K.SUJITH

2303A52346

BATCH 45

Ass 5.5

### Task Description #1 (Transparency in Algorithm Optimization)

Task: Use AI to generate two solutions for checking prime numbers:

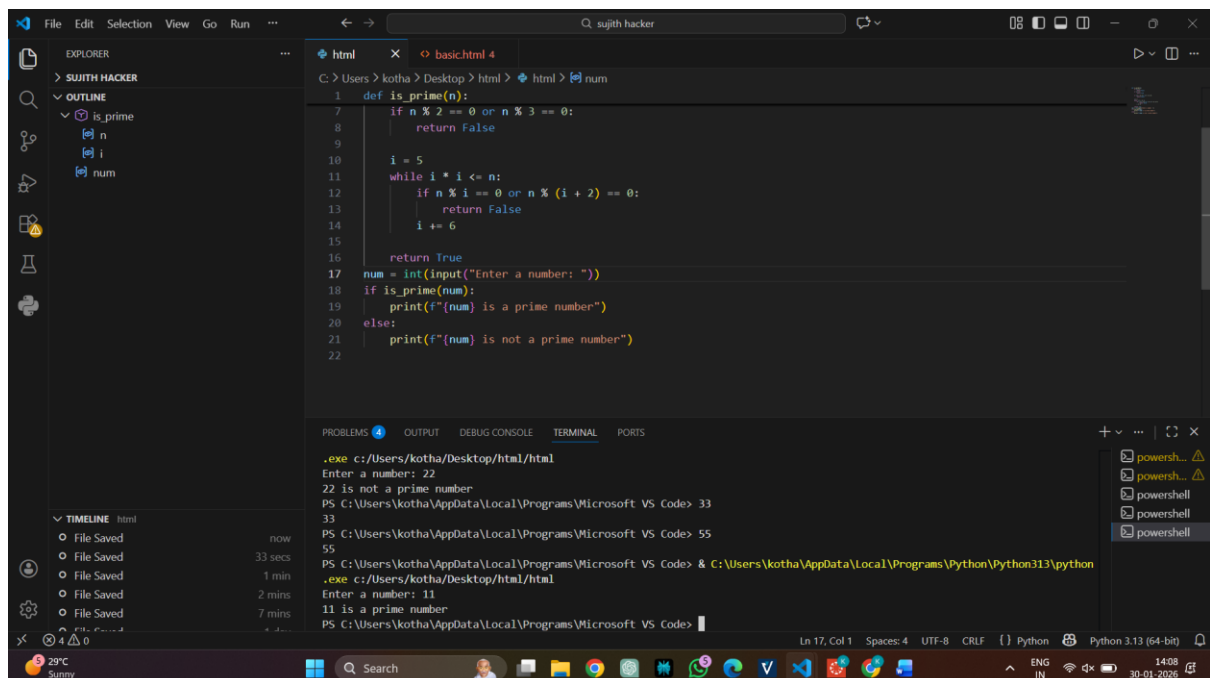
- Naive approach(basic)
- Optimized approach

Prompt:

“Generate Python code for two prime-checking methods and explain how the optimized version improves performance.”

Expected Output:

- Code for both methods.
- Transparent explanation of time complexity.
- Comparison highlighting efficiency improvements



The screenshot shows a Visual Studio Code editor with a Python file named `basic.html 4`. The code defines a function `is_prime(n)` and uses it to check if a user-input number is prime. The function first checks for divisibility by 2 and 3, then uses a while loop for other primes. The terminal window shows the execution of the script, with the user entering 22 and 11, and the program outputting whether each is a prime number.

```
def is_prime(n):
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True

num = int(input("Enter a number: "))
if is_prime(num):
    print(f"{num} is a prime number")
else:
    print(f"{num} is not a prime number")
```

Terminal Output:

```
.exe c:/Users/kotha/Desktop/html/html
Enter a number: 22
22 is not a prime number
PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code> 33
33
PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code> 55
55
PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kotha\AppData\Local\Programs\Python\Python313\python
.exe c:/Users/kotha/Desktop/html/html
Enter a number: 11
11 is a prime number
PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code>
```

### Task Description #2 (Transparency in Recursive Algorithms)

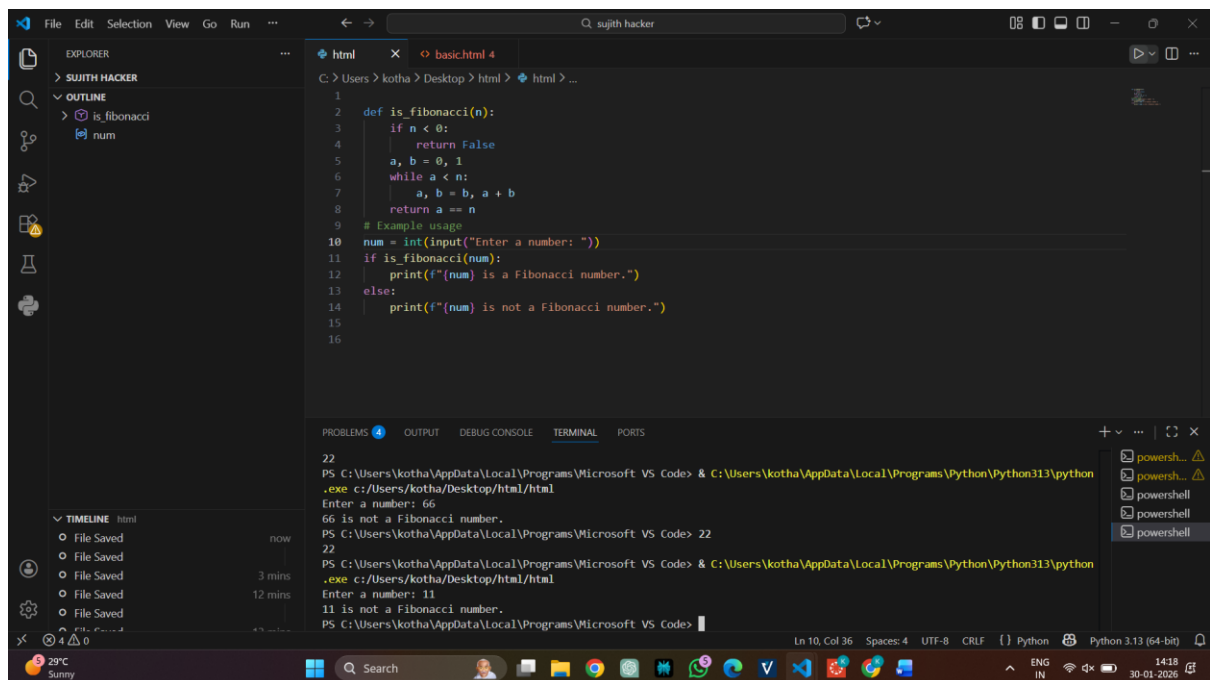
Objective: Use AI to generate a recursive function to calculate Fibonacci numbers.

Instructions:

1. Ask AI to add clear comments explaining recursion.
2. Ask AI to explain base cases and recursive calls.

Expected Output:

- Well-commented recursive code.
- Clear explanation of how recursion works.
- Verification that explanation matches actual execution.



```
1
2 def is_fibonacci(n):
3     if n < 0:
4         return False
5     a, b = 0, 1
6     while a < n:
7         a, b = b, a + b
8     return a == n
9 # Example usage
10 num = int(input("Enter a number: "))
11 if is_fibonacci(num):
12     print(f"{num} is a Fibonacci number.")
13 else:
14     print(f"{num} is not a Fibonacci number.")
15
16
```

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
22 PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kotha\AppData\Local\Programs\Python\Python313\python
.exe c:/Users/kotha/Desktop/html/html
Enter a number: 66
66 is not a Fibonacci number.
22 PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code>
PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kotha\AppData\Local\Programs\Python\Python313\python
.exe c:/Users/kotha/Desktop/html/html
Enter a number: 11
11 is not a Fibonacci number.
PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code>
```

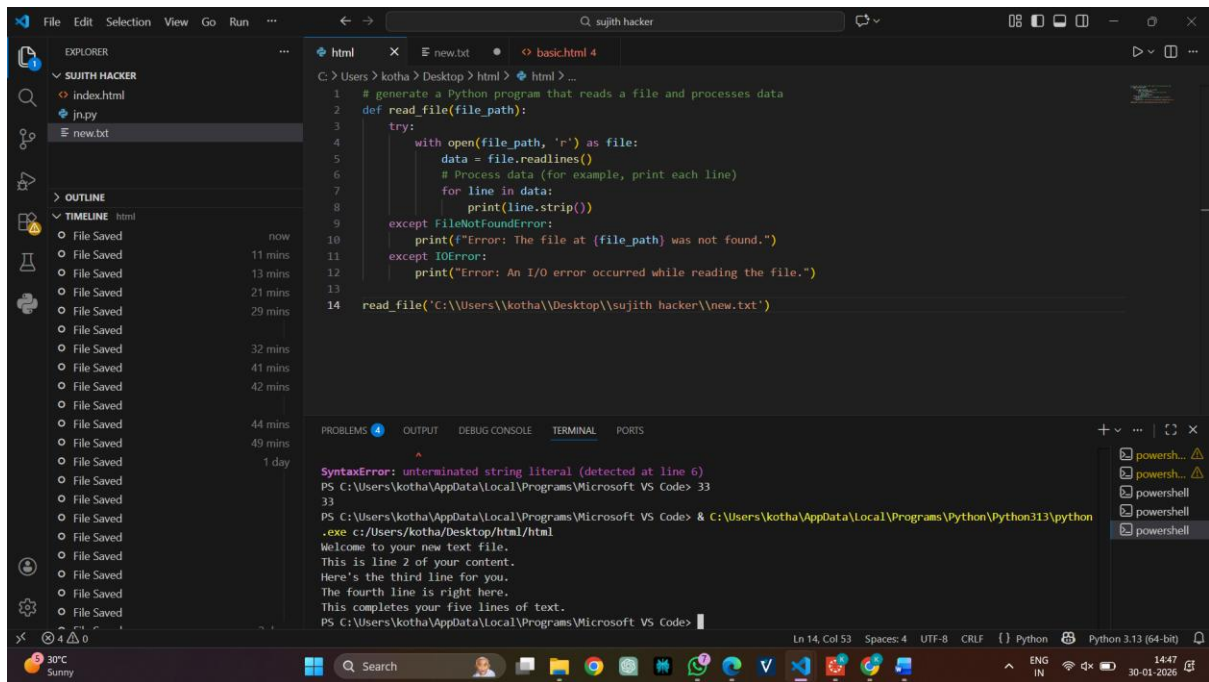
Use AI to generate a Python program that reads a file and processes data.

Prompt:

“Generate code with proper error handling and clear explanations for each exception.”

Expected Output:

- Code with meaningful exception handling.
- Clear comments explaining each error scenario.
- Validation that explanations align with runtime behavior.

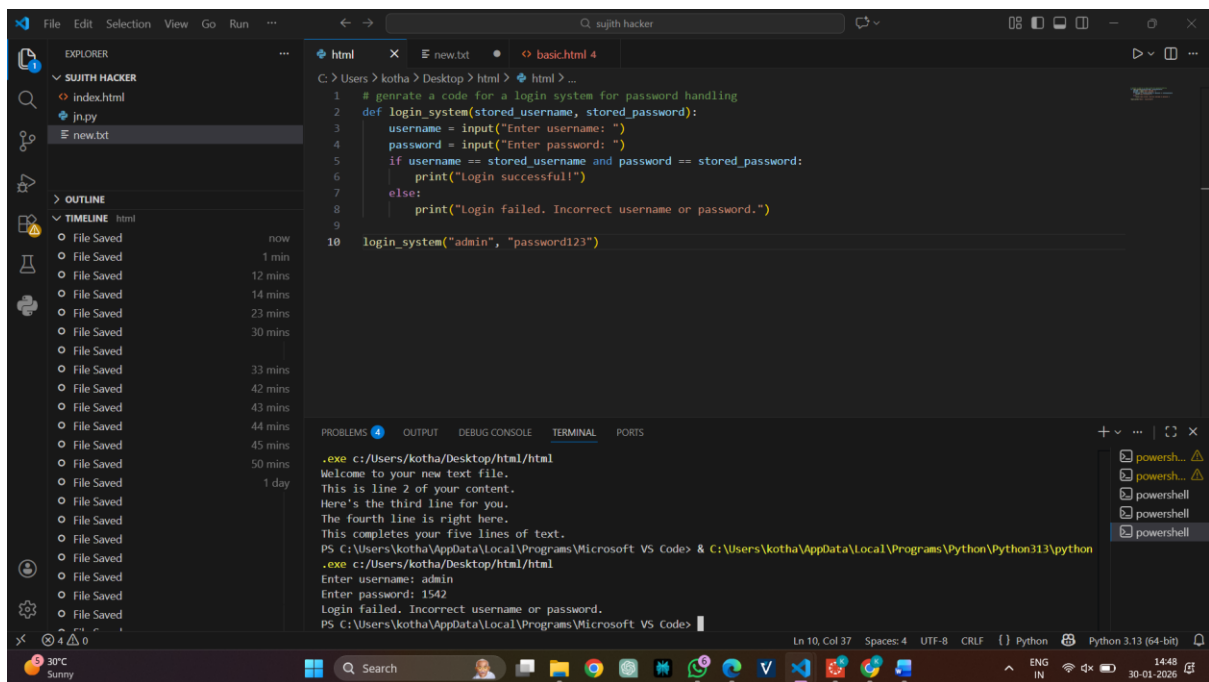


Use an AI tool to generate a Python-based login system.

Analyze: Check whether the AI uses secure password handling practices.

Expected Output:

- Identification of security flaws (plain-text passwords, weak validation).
- Revised version using password hashing and input validation



Use an AI tool to generate a Python script that logs user

activity (username, IP address, timestamp).

Analyze: Examine whether sensitive data is logged unnecessarily or insecurely.

Expected Output:

- Identified privacy risks in logging.
- Improved version with minimal, anonymized, or masked logging.
- Explanation of privacy-aware logging principles.

The screenshot shows a Visual Studio Code editor window with a Python script titled 'html' open. The script is designed to log user activity, comparing a basic logging method with a privacy-aware method. The basic method logs the full IP address and timestamp, while the privacy-aware method masks the last two octets of the IP address and uses a more concise timestamp format. The script includes comments explaining the privacy risks and the improvements made. The Explorer sidebar on the left shows a file tree with 'index.html' and 'new.txt'. The Timeline sidebar on the right shows a list of file save events. The bottom status bar indicates the current line and column (Ln 17, Col 61) and the active Python interpreter (Python 3.13 (64-bit)).

```
1 # Generate a Python script that logs user activity (username, IP address, timestamp), identify privacy risks in the
2 import logging
3 from datetime import datetime
4 # Basic logging of user activity (privacy risks: storing IP address and exact timestamp)
5 def log_user_activity(username, ip_address):
6     logging.basicConfig(filename='user_activity.log', level=logging.INFO)
7     timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
8     logging.info(f"User: {username}, IP: {ip_address}, Time: {timestamp}")
9 # Improved privacy-aware logging (anonymized IP address and date only)
10 def log_user_activity_privacy_aware(username, ip_address):
11     logging.basicConfig(filename='user_activity_privacy.log', level=logging.INFO)
12     date = datetime.now().strftime("%Y-%m-%d")
13     anonymized_ip = '.'.join(ip_address.split('.')[:2]) + '.xxx.xxx' # Masking last two octets
14     logging.info(f"User: {username}, IP: {anonymized_ip}, Date: {date}")
15 # Explanation: The improved version masks the last two octets of the IP address to protect user identity and logs on
16 log_user_activity("user1", "2514789192.168.1")
17 log_user_activity_privacy_aware("user1", "2514789192.168.1")
```